MODULE 04 · DOMAIN 1 - MONITORING, LOGGING, ANALYSIS, REMEDIATION, AND PERFORMANCE OPTIMIZATION

Reading Logs at Scale

Metric filters, subscription filters and Logs Insights. Three routes out of a log group, and the exam tests which one you pick.

DOMAIN 1 of 5 EXAM WEIGHT 22% MODULE 4 of 35

admin@sthithapragna.com

Ask what the log is **for**. Alarm on it, ship it somewhere, or interrogate it. Three questions, three mechanisms.

The three routes out of a log group

MECHANISM	WHAT IT GIVES YOU
Metric filter	A CloudWatch metric. Alarm on it, graph it, use it in metric math
Subscription filter	The events themselves, streamed in near real time, to another service
Logs Insights	Interactive queries. Answers, not automation
Export to S3	A batch copy for archive or Athena. Not real time
Live Tail	A streaming view for a human, right now. Nothing is stored

ON THE EXAM

Metric filters and subscription filters both need the Standard log class. Infrequent Access has neither.

Metric filters: text becomes a number

- A **pattern** plus a **metric transformation**: which metric to publish, what value, and what dimensions.
- Value can be a **literal 1** to count occurrences, or a **field from the log event** such as a duration or a byte count.
- **Not retroactive.** A filter created today never sees yesterday's events, no matter how long they are retained.
- **100 metric filters per log group**, and that limit cannot be raised.

> ON THE EXAM
>
> Filters run at ingestion, exactly like data protection policies. Anything already in the log group is out of reach.

The pattern syntax, in three shapes

- **Plain terms.** **ERROR** matches events containing it. **"ERROR" "Timeout"** requires both. **?ERROR ? WARN** matches either.
- **Exclusion** with a minus: **ERROR -Timeout** matches errors that are not timeouts.
- **JSON** events use dollar-dot notation: **{ $.statusCode = 500 }**. Numeric and string comparisons both work.
- **Space-delimited** events use bracket notation with positional names: **[ip, user, ..., status=404]**.

> ON THE EXAM
>
> Structured logs make filters simple. If you control the application, log JSON and the pattern becomes a field comparison.

The default value trap

- With **no default value set**, a period with no matching events publishes **nothing at all**.
- The alarm then sees missing data and goes to **INSUFFICIENT_DATA**, not OK. Everything is fine and the alarm looks broken.
- Set **default value 0** and the filter publishes a zero for quiet periods, so the alarm sits properly in OK.
- Or leave it unset and set the alarm's missing-data treatment to **notBreaching**. Either works. Doing neither is the bug.

> ON THE EXAM
>
> This is module two's missing-data lesson with a specific cause. "Our error alarm is always in INSUFFICIENT_DATA" is this, almost every time.

Metric filter or Logs Insights?

METRIC FILTER WHEN

You know what to look for

- You want an alarm, so you need a metric
- The pattern is known in advance
- Costs nothing extra at query time
- Only sees events after you created it

LOGS INSIGHTS WHEN

You are working it out

- Investigating, not automating
- Ad hoc questions you did not predict
- Can query the full retained history
- Charged per byte scanned, per query

THE TIEBREAKER

Filters look forward, queries look backward. That single sentence answers most questions in this area.

Subscription filters: events on the move

- Streams matching events in **near real time** to **Lambda**, **Kinesis Data Streams**, **Amazon Data Firehose** or **OpenSearch Service**.
- **Lambda** for custom processing and routing. **Firehose** for managed delivery to S3, OpenSearch or Redshift.
- **Kinesis Data Streams** when several independent consumers need the same events.
- The same filter pattern syntax applies, so you can stream only what matters instead of everything.

> ON THE EXAM
>
> Firehose buffers, so expect delivery in minutes. Lambda is the low-latency option when seconds matter.

Two limits that shape designs

- **Two subscription filters per log group.** Firm, and it cannot be raised.
- Need more destinations? **Fan out through Kinesis Data Streams** and attach consumers to that instead.
- Cross-account streaming uses a **destination** in the receiving account, with an access policy naming the sending accounts.
- The sending side needs an **IAM role CloudWatch Logs can assume** to write to the destination.

> ON THE EXAM
>
> "We already stream to a SIEM and now the security team wants a copy too" is a two-filter problem. The third destination goes behind Kinesis.

Subscription filter or export to S3?

SUBSCRIPTION FILTER WHEN

Continuous and current

- Near real time, event by event
- Feeds a SIEM, an index, or a processing pipeline
- Filter pattern selects what leaves
- Always running once configured

EXPORT TO S3 WHEN

A bulk copy, on request

- CreateExportTask, a batch job over a time range
- Can take up to 12 hours to finish
- For archive, Athena, or handing data to someone
- One export task at a time per account

THE TIEBREAKER

"Continuously", "as they occur" or "near real time" means subscription filter. "Archive" or "analyse in Athena" means export.

From a log line to a page

01
Event arrives
An application writes a line to the log group.

02
Transformer
Optional. Parses it into real fields at ingest.

03
Metric filter
Pattern matches, publishes a metric data point.

04
Alarm
M out of N over the metric, missing data handled.

05
SNS or action
Notify, invoke Lambda, raise an OpsItem.

ON THE EXAM

Five links, and a break in any one produces the same symptom: nobody was told. Troubleshoot along the chain, not at the ends.

Logs Insights: the commands you actually use

- **fields** chooses columns, **filter** narrows rows, **sort** and **limit** finish the job.
- **stats** aggregates: **stats count(*) by bin(5m)** is the single most useful query shape there is.
- **parse** pulls fields out of unstructured text. **dedup** collapses repeats.
- **JOIN and sub-query** arrived in April 2026, so correlating two log groups no longer needs OpenSearch SQL.

> ON THE EXAM
>
> stats … by bin() turns a wall of text into a shape over time. Learn that one query and half of incident triage gets faster.

Three query languages, one console

LANGUAGE	REACH FOR IT WHEN
Logs Insights QL	The default. Fewest commands, works with existing saved queries and dashboards
OpenSearch PPL	Pipe-and-search style. Familiar if the team already uses OpenSearch
OpenSearch SQL	Your team already writes SQL, or you are porting existing OpenSearch queries

ON THE EXAM

Since April 2026 Logs Insights QL has JOIN and sub-query too, so the languages differ mainly by familiarity rather than capability.

Current quotas, and the ones everyone still quotes

LIMIT	CURRENT VALUE
Concurrent Logs Insights QL queries	100 per account. Was 30 until March 2026
Concurrent OpenSearch SQL or PPL queries	15 per account
Log groups in one query	50 by name, or up to 10,000 via SOURCE with a prefix
Query timeout	60 minutes. The old 15-minute figure is long dead
Results returned	100,000 with LIMIT. Was 10,000 until May 2026
Query results kept	7 days

ON THE EXAM

Study guides written before 2026 say 20 log groups, 10 concurrent queries and 10,000 results. All three are wrong now.

What Logs Insights costs, and how to spend less

- You are charged for **bytes scanned**, so a narrow time range is the cheapest optimisation there is.
- **Field indexes** let the engine skip events that cannot match. Use **filterIndex** to query only indexed log groups.
- Put **filter** early in the pipeline, before **stats**. Reduce rows before you aggregate them.
- A dashboard widget running a Logs Insights query **re-runs and re-charges** every time the dashboard loads.

> ON THE EXAM
>
> The dashboard point surprises teams every month. A Logs Insights widget on a wall-mounted screen is a standing charge.

Two patterns the exam expects you to recognise

- **CloudTrail to CloudWatch Logs to metric filter to alarm.** The standard way to alert on API activity such as root account use.
- This is how CIS-style controls are implemented. CloudTrail alone **records**; it does not alert.
- **Contributor Insights** builds top-N rules over structured logs: the noisiest IP, the busiest tenant, the heaviest caller.
- Contributor Insights answers **who** is causing the load. A metric filter only tells you **how much** there is.

> ON THE EXAM
>
> "Alert when the root user signs in" is always CloudTrail into a log group, then a metric filter, then an alarm. Not CloudTrail on its own.

A team creates a metric filter that counts the string PaymentDeclined in an application log group, publishing a value of 1 for each match. They build an alarm on it with a five-minute period and a threshold of more than 10.

Declines are rare. The alarm has been in INSUFFICIENT_DATA almost continuously since it was created, and it did not fire during an incident last week when 40 declines occurred in ten minutes.

What explains both symptoms, and what is the fix?

A The metric filter is not retroactive, so it missed the incident. Recreate it

B No default value is set, so quiet periods publish nothing. Set the default value to 0

C The log group is in the Infrequent Access class. Recreate it as Standard

D The alarm period is shorter than the publishing interval. Increase it to 15 minutes

◆ PAUSE AND DECIDE

What explains both symptoms, and what is the fix?

A Not retroactive

True of metric filters, but irrelevant. The filter existed before the incident, so it did see those events.

B Set default value to 0

Explains the constant INSUFFICIENT_DATA, and that state is why the alarm could not evaluate normally during the burst.

C Infrequent Access class

Self-defeating. An IA log group cannot have a metric filter at all, so the filter would never have been created.

D Period too short

The module two trap, applied to the wrong cause. The metric does not publish on an interval; it publishes when something matches.

THE SIGNAL

One cause, both symptoms. A metric that only exists when something goes wrong leaves the alarm blind the rest of the time.

An application log group already has two subscription filters: one to a Lambda function that redacts and forwards events to a partner, and one to Amazon Data Firehose delivering to an S3 data lake.

The security team now needs the same events delivered to their SIEM in near real time. An engineer tries to add a third subscription filter and the request fails.

What should the engineer do?

A Request a service quota increase for subscription filters per log group

B Replace one filter with a Kinesis Data Streams destination and attach the existing consumer plus the SIEM to that stream

C Schedule a CreateExportTask to S3 every five minutes and have the SIEM read the bucket

D Create a second log group and configure the application to write every event to both

◆ PAUSE AND DECIDE

What should the engineer do?

A Request a quota increase

The two-filter limit is firm and not adjustable. A plausible-sounding action that AWS will not grant.

B Fan out through Kinesis Data Streams

One filter feeds a stream; any number of consumers read it independently. The documented answer to needing more than two destinations.

C Scheduled export to S3

Export is a batch task that can take hours and runs one at a time per account. It cannot be near real time.

D Duplicate log group

Doubles ingestion cost, needs an application change, and creates two versions of the truth that will drift.

THE SIGNAL

When a hard limit blocks you, the answer is a different topology, not a bigger number. Fan-out is that topology.

During an incident, a CloudOps engineer needs to find which API endpoints returned 5xx responses across 38 microservice log groups, and when the errors started. The log groups all share the name prefix /aws/apps/prod/.

The engineer has no metric filters for this and needs an answer in the next few minutes, covering the last six hours.

What is the fastest correct approach?

A Create a metric filter on each log group for 5xx responses and wait for the metrics to populate

B Run a Logs Insights query using SOURCE with the log group prefix, filtering on status and using stats count by bin

C Export all 38 log groups to S3 and query the results with Amazon Athena

D Open Live Tail on all 38 log groups and watch for 5xx responses

◆ PAUSE AND DECIDE

What is the fastest correct approach?

A. Create 38 metric filters

Filters are not retroactive. They would show nothing about the last six hours, which is the entire question.

B. Logs Insights with SOURCE prefix

Queries retained history immediately, SOURCE avoids naming 38 groups, and stats by bin shows when it started.

C. Export to S3, query in Athena

Correct eventually. Export can take hours and runs one task at a time, which is useless during an incident.

D. Live Tail everywhere

Live Tail shows events arriving now. It cannot look back six hours, and it cannot aggregate.

THE SIGNAL

"Looking backward" plus "right now" is always Logs Insights. Filters look forward and exports are slow.

A compliance requirement states that the security team must be notified within minutes whenever the root user signs in to any of the company's accounts. CloudTrail is already enabled as an organization trail delivering to an S3 bucket in the security account.

The security team argues that because CloudTrail is enabled, the requirement is already met.

What must still be implemented?

A Nothing. An organization trail already records root sign-ins

B Configure the trail to also deliver to CloudWatch Logs, add a metric filter for root sign-in, and alarm on it with an SNS action

C Enable S3 event notifications on the CloudTrail bucket to notify the security team

D Schedule an Athena query over the CloudTrail bucket every morning and email the results

◆ PAUSE AND DECIDE

What must still be implemented?

A. Nothing, CloudTrail records it

Recording is not alerting. The event is captured in a bucket nobody is watching, which satisfies audit and not notification.

B. Trail to CloudWatch Logs, metric filter, alarm

The standard chain. Log group makes the events filterable, the filter makes a metric, the alarm makes it a notification.

C. S3 event notifications

Fires when a log file object is written, which happens constantly. It says a file arrived, never what is inside it.

D. Daily Athena query

Right data, wrong latency. The requirement says within minutes and this answers within a day.

THE SIGNAL

Detective controls have two halves: capture and notify. CloudTrail is capture. Nothing in CloudTrail alone tells a human anything.

The five things to carry forward

- **Filters look forward, queries look backward.** Neither one substitutes for the other.
- **Set a default value of 0** on counting metric filters, or the alarm lives in INSUFFICIENT_DATA.
- **Two subscription filters per log group, firm.** Need more destinations, fan out through Kinesis Data Streams.
- **Subscription filter is continuous; export to S3 is a slow batch.** "Near real time" decides it.
- **CloudTrail records, it does not alert.** The chain is trail, log group, metric filter, alarm.

> ON THE EXAM
>
> Next: CloudTrail itself, and the difference between who did it, what it looked like, and how it is performing.